REAL ESTATE MANAGEMENT SYSTEM

Project Documentation

Developed using Java, Spring Boot and MySQL

Prepared by: **THAVITHU AROCKIA KEIN**

Table of Contents

1. Project Overview
2. Objectives
3. User Roles and Responsibilities
4. Technologies Used
5. System Architecture
6. Project Structure
7. Main Entities
8. Authentication and Security
9. Main Functional Modules
10. Business Rules, Validation, and Error Handling
11. REST API Documentation
12. Typical System Workflow
13. Example API Requests
14. Scheduled Tasks and Automation
15. Postman Testing Order
16. Common Errors and Troubleshooting
17. Project Screenshots
18. Future Enhancements
19. Project Summary

1. Project Overview

The Real Estate Management System is a backend REST API application developed using Java and Spring Boot. It is designed to organize key rental-property operations, including property listings, property owners, tenants, lease agreements, rent payments, and maintenance requests. The application uses MySQL for persistent storage and Spring Data JPA/Hibernate for database interaction. It also includes JWT-based authentication, request validation, centralized exception handling, reporting endpoints, and scheduled tasks for lease and payment management.

2. Objectives

- Maintain property details, including type, location, area, bedrooms, bathrooms, and rent.
- Manage property-owner information.
- Manage tenant information.
- Create, view, update, and terminate lease agreements.
- Record rent-payment information and mark payments as paid or overdue.
- Create and track property maintenance requests.
- Provide filtering, pagination, and reporting endpoints for selected resources.
- Authenticate users using JWT and apply configured access rules.
- Validate incoming data and return consistent error responses.
- Automate selected lease-expiry and overdue-payment tasks using scheduled jobs.

3. User Roles and Responsibilities

The project defines application roles through a Role enum and includes registration/login endpoints with JWT authentication. The precise permissions for each role are configured in the security layer; verify these rules against the current SecurityConfig before presenting role-specific access as a final claim.

Role / user type	Typical responsibility in the system
Administrator	Manage and oversee property, owner, tenant, lease, payment, and maintenance information, subject to configured authorization.
Authenticated user	Sign in and access endpoints permitted by the configured security rules.

4. Technologies Used

Technology	Purpose
Java 25	Application development language (as configured in pom.xml).
Spring Boot 4.1.1	Application framework and runtime configuration.
Spring Web MVC	REST API controllers and HTTP request/response handling.
Spring Data JPA / Hibernate	Object-relational mapping and database persistence.
MySQL	Relational database; configured database name: realestate_db.
Spring Security	Authentication and endpoint access control.
JWT (JJWT 0.12.6)	Token-based authentication.
Bean Validation	Validation of request DTOs.
Maven	Dependency management and build lifecycle.
Springdoc OpenAPI 3.0.3	API documentation interface.
Lombok and MapStruct	Reduce boilerplate and map between entities and DTOs.
Spring Actuator	Health and metrics endpoints.
Spring Cache	Caching support configured in the application.

5. System Architecture

The application follows a layered backend architecture. Requests enter through REST controllers, business operations are handled by service classes, and repositories interact with the MySQL database. DTOs are used for request/response data, while mapper classes convert between entities and DTOs.

Client / Postman / API consumer

↓

REST Controller Layer

↓

Service Layer (business rules)

↓

Repository Layer (Spring Data JPA)

↓

MySQL Database

Supporting components: Security/JWT • DTOs and Mappers • Validation and Global Exception Handler • Scheduler • OpenAPI

6. Project Structure

Package / component	Responsibility
config	Security, caching, auditing, startup data, OpenAI configuration, and scheduler configuration.
controller	REST endpoints for authentication, owners, properties, tenants, leases, maintenance requests, and payments.
service / service.impl	Business operations and service implementations.
repository	Database access using Spring Data repositories.
entity	JPA entities representing users, owners, properties, tenants, leases, payments, and maintenance requests.
dto	Request and response objects, page responses, and report summaries.
mapper	Entity-to-DTO and DTO-to-entity mapping.
security	JWT service, authentication filter, user-details service, and security error handlers.

exception	Custom exceptions and centralized exception response handling.
specification	Dynamic filtering/search specifications for selected resources.
scheduler	Scheduled real-estate maintenance tasks.
enums	Role, property type/status, lease status, maintenance priority/status, and payment method/status values.

7. Main Entities

Entity	Purpose / representative data
AppUser	Login account information such as username and password hash.
Owner	Property owner details such as name, phone, email, and address.
Property	Property details such as owner, title, description, type, address, city, bedrooms, bathrooms, area, and monthly rent.
Tenant	Tenant details such as name, phone, email, address, and occupation.
Lease	Links a property and tenant; stores start/end dates, monthly rent, and security deposit.
RentPayment	Payment record linked to a lease, including amount, due date, paid date, and payment method.
MaintenanceRequest	Maintenance issue linked to a property and tenant, with title, description, priority, cost, and completion time.
BaseEntity	Common created-at and updated-at audit timestamps.

Important relationships: one owner can be associated with multiple properties; a property can be referenced by lease and maintenance records; a tenant can have lease

and maintenance records; a lease can have multiple rent-payment records. Exact cardinality and cascade behavior are defined by the entity annotations.

8. Authentication and Security

The project includes registration and login endpoints under /api/auth and uses Spring Security with JWT-related components. After successful login, the API returns an authentication response containing token information. Protected requests should include the token in the Authorization header using the Bearer scheme, when required by the active security configuration.

Example header: Authorization: Bearer <JWT_TOKEN>

Security note: Never publish actual database passwords, administrator credentials, JWT secrets, or live tokens in the project report. Replace local development secrets with environment variables or secure deployment configuration before production use.

9. Main Functional Modules

9.1 Authentication

- Register an account and log in through /api/auth/register and /api/auth/login.

9.2 Owner Management

- Create, retrieve, list, update, and delete owner records.

9.3 Property Management

- Create, retrieve, search/filter, update, change property status, and delete properties.
- Retrieve properties by owner and access city statistics and status-count reports.

9.4 Tenant Management

- Create, retrieve, list, update, and delete tenant records.

9.5 Lease Management

- Create and view lease records, list leases with pagination/filtering, update lease details, terminate a lease, and retrieve expiring leases.
- An endpoint is available to process overdue lease expiry; scheduled expiry processing is also configured.

9.6 Rent Payment Management

- Create and view payment records, list payments, mark a payment as paid, view a lease payment summary, and process overdue payments.

9.7 Maintenance Requests

- Create and view maintenance requests, list requests, update request status, and delete a request.

9.8 Reporting and Search

- Property reporting includes city statistics and property status counts. Pagination and dynamic specifications are implemented for selected listing endpoints.

10. Business Rules, Validation, and Error Handling

- Controller request DTOs use Bean Validation where declared.
- Custom exceptions include resource-not-found, duplicate-resource, business-rule, and invalid-credentials exceptions.
- GlobalExceptionHandler centralizes API exception responses.
- Enum types are used for property type/status, lease status, maintenance priority/status, payment method/status, and user roles.
- Lease expiry and overdue payment handling are supported through service/controller operations and scheduled tasks.

The exact business rules (for example, whether overlapping leases are prevented, or which payment transitions are permitted) should be described only after confirming the corresponding service implementation and current test results.

11. REST API Documentation

The following endpoints are identified in the project's controllers. Use the application's configured host and port; the current configuration uses port 8080. Authentication requirements depend on SecurityConfig.

Method	Endpoint	Purpose
POST	/api/auth/register	Register an account.
POST	/api/auth/login	Authenticate and receive token response.
POST	/api/owners	Create an owner.

GET	/api/owners/{id}	Get owner by ID.
GET	/api/owners	List owners (paged).
PUT	/api/owners/{id}	Update owner.
DELETE	/api/owners/{id}	Delete owner.
POST	/api/properties	Create a property.
GET	/api/properties/{id}	Get property by ID.
GET	/api/properties	Search/list properties.
GET	/api/properties/owner/{ownerId}	List properties belonging to an owner.
PUT	/api/properties/{id}	Update property.
PATCH	/api/properties/{id}/status	Update property status.
DELETE	/api/properties/{id}	Delete property.
GET	/api/properties/reports/city-stats	Get property statistics by city.
GET	/api/properties/reports/status-count	Get property counts by status.
POST	/api/tenants	Create a tenant.
GET	/api/tenants/{id}	Get tenant by ID.
GET	/api/tenants	List tenants (paged).
PUT	/api/tenants/{id}	Update tenant.
DELETE	/api/tenants/{id}	Delete tenant.
POST	/api/leases	Create a lease.
GET	/api/leases/{id}	Get lease by ID.
GET	/api/leases	List leases (paged/filterable).
PUT	/api/leases/{id}	Update lease.
PATCH	/api/leases/{id}/terminate	Terminate a lease.
GET	/api/leases/expiring	Get expiring leases.
POST	/api/leases/expire-overdue	Process overdue lease expiry.
POST	/api/payments	Create a rent-payment record.
GET	/api/payments/{id}	Get payment by ID.
GET	/api/payments	List payments (paged/filterable).
PATCH	/api/payments/{id}/pay	Mark a payment as paid.
DELETE	/api/payments/{id}	Delete a payment

		record.
GET	/api/payments/summary?leaseId={id}	Get payment summary for a lease.
POST	/api/payments/mark-overdue	Process overdue payments.
POST	/api/maintenance-requests	Create a maintenance request.
GET	/api/maintenance-requests/{id}	Get maintenance request by ID.
GET	/api/maintenance-requests	List maintenance requests (paged/filterable).
PATCH	/api/maintenance-requests/{id}/status	Update maintenance status.
DELETE	/api/maintenance-requests/{id}	Delete a maintenance request.

12. Typical System Workflow

User registers / logs in

↓

JWT token returned (if login succeeds)

↓

Create owner and property records

↓

Register tenant details

↓

Create lease linking property and tenant

↓

Create rent-payment records for the lease

↓

Record payments / review payment summary

↓

Create maintenance request when needed

↓

Track lease status, upcoming expiries, and reports

13. Example API Requests

13.1 Login

```
POST http://localhost:8080/api/auth/login
Content-Type: application/json

{
  "username": "your-username",
  "password": "your-password"
}
```

The exact request fields and response structure are defined by LoginRequestDTO and AuthResponseDTO. Use those classes as the source of truth if the sample payload differs from your local implementation.

13.2 Create a Property

```
POST http://localhost:8080/api/properties
Content-Type: application/json
Authorization: Bearer <JWT_TOKEN>
{
  "ownerId": 1,
  "title": "Sample Apartment",
  "description": "Two-bedroom apartment",
  "propertyType": "APARTMENT",
  "address": "Sample Street",
  "city": "Chennai",
  "bedrooms": 2,
  "bathrooms": 2,
  "areaSqft": 950,
  "monthlyRent": 18000
}
```

This is an illustrative request body based on PropertyRequestDTO field names; required fields and accepted enum values should be checked against the DTO and enum definitions.

13.3 Mark a Rent Payment as Paid

```
PATCH http://localhost:8080/api/payments/1/pay
Content-Type: application/json
Authorization: Bearer <JWT_TOKEN>
{ /* use the fields declared in RentPaymentRequestDTO */}
```

14. Scheduled Tasks and Automation

Scheduler configuration is present and enabled by default in application.properties. The configured tasks include lease-expiry processing, overdue-payment marking, and lease-expiry reminders. The configured cron schedules are: lease expiry at 01:00, overdue payments at 01:30, and expiry reminders at 09:00 (server-local scheduling behavior depends on runtime configuration).

A manual endpoint is also available to process overdue lease expiry and another to mark overdue rent payments. Verify scheduler logs and service behavior during testing.

15. Postman Testing Order

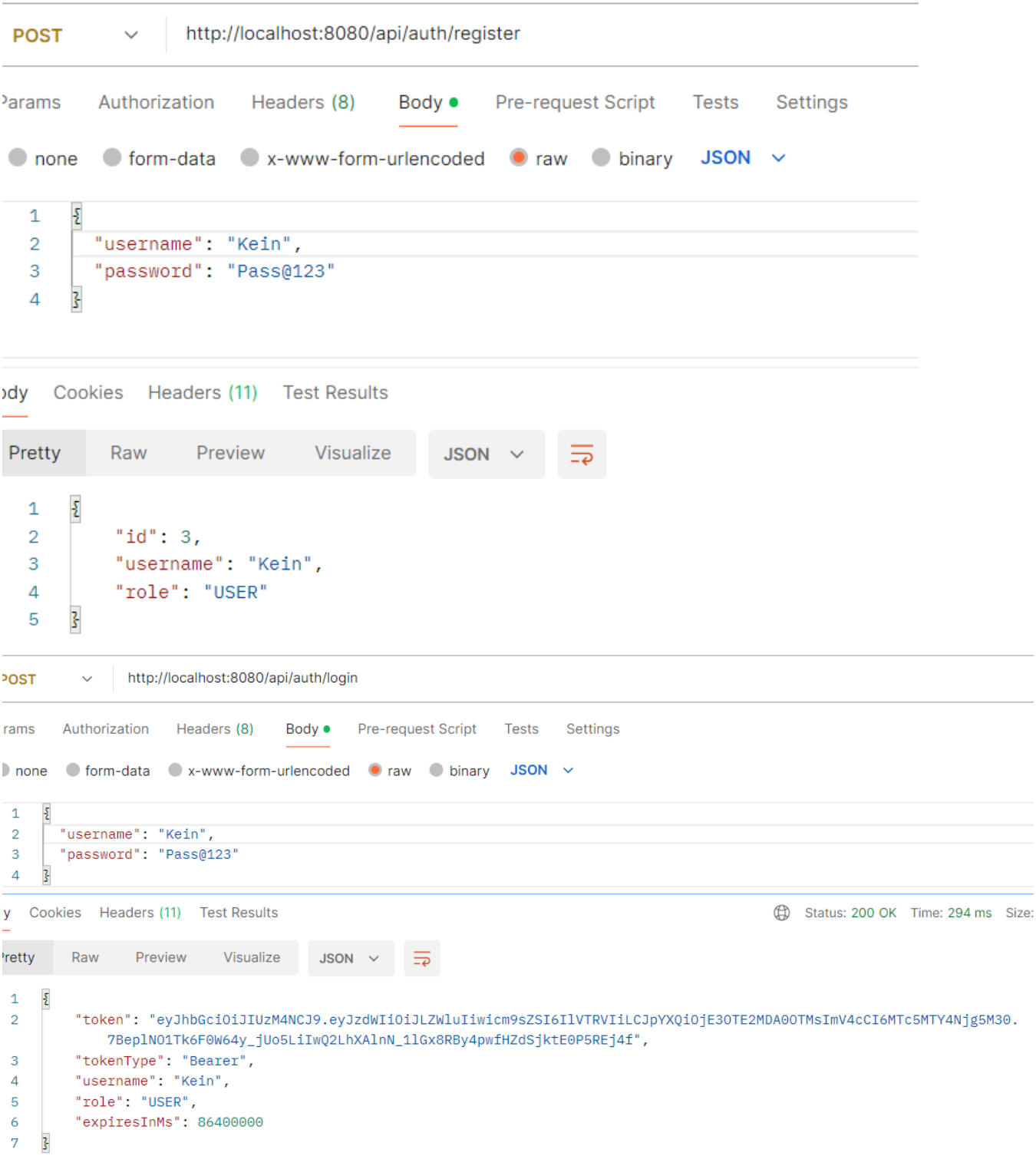
1. Start MySQL and confirm the realestate_db datasource settings.
2. Run the Spring Boot application on port 8080.
3. Register an account using POST /api/auth/register.
4. Log in using POST /api/auth/login and copy the returned token if required.
5. Create an owner and create a property associated with that owner.
6. Create a tenant.
7. Create a lease linking an existing property and tenant.
8. Create a rent-payment record for the lease and test the pay endpoint.
9. Create a maintenance request and update its status.
10. Test list/search endpoints, pagination, and property reports.
11. Test not-found, validation, duplicate, and unauthorized/forbidden responses.

16. Common Errors and Troubleshooting

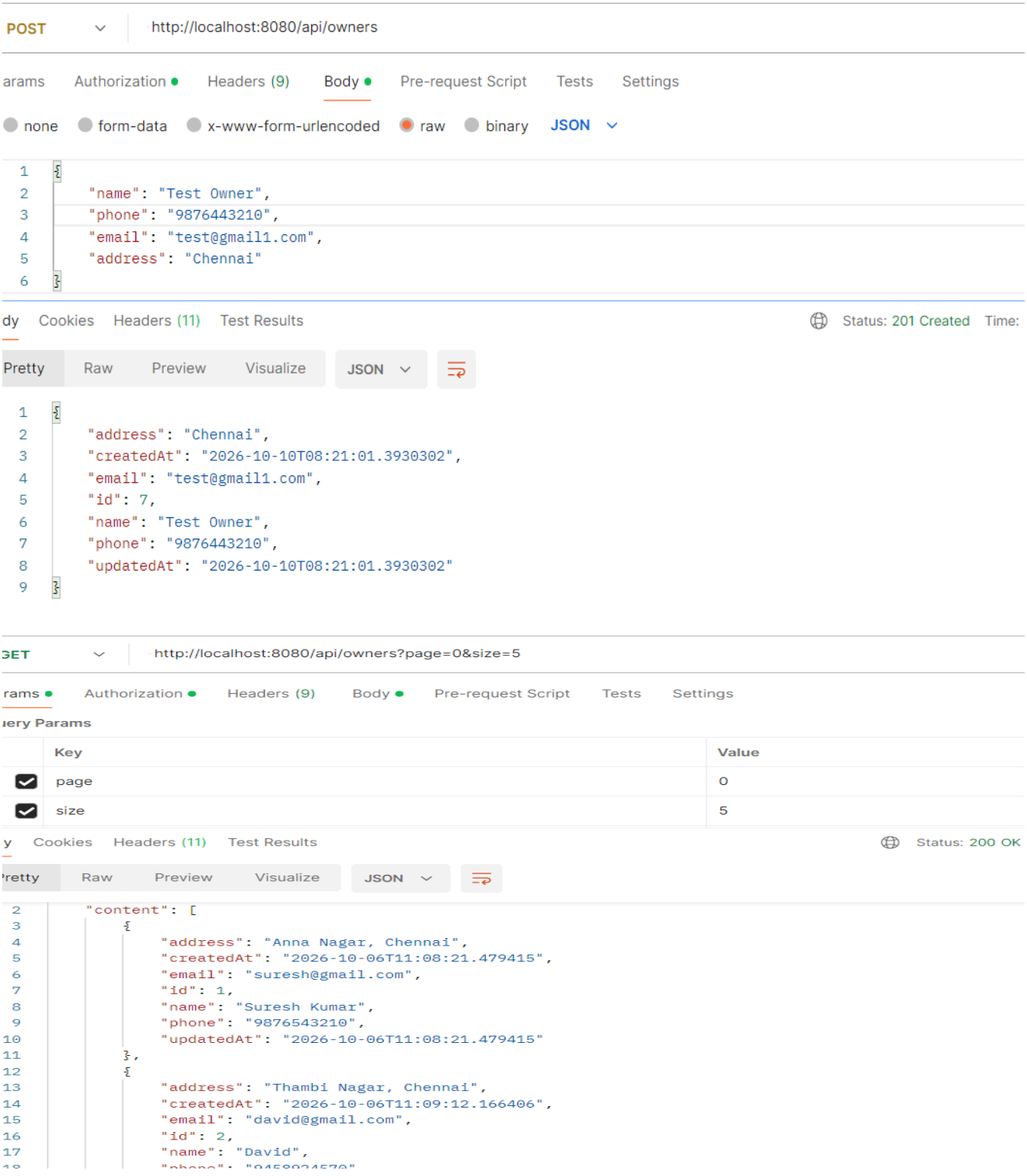
Issue	What to check
401 Unauthorized	Log in again and send a valid Bearer token if the endpoint is protected.
403 Forbidden	Check the role/authorization rules in SecurityConfig.
404 Not Found	Verify the HTTP method, base URL, endpoint path, and resource ID.
500 Internal Server Error	Read the Spring Boot console stack trace; check request JSON, service logic, entity relationships, and database constraints.
MySQL connection failure	Confirm MySQL is running and the database username/password and URL are correct.
Validation error	Check required DTO fields, field formats, and enum values.
Unexpected paging/filter results	Review query parameters and the relevant Specification/controller defaults.
Scheduler not running	Confirm app.scheduler.enabled is true and inspect startup/log output.

17. Project Screenshots

12. Registration and login request/response in Postman.



13. Owner creation and owner listing.



14. Property creation and property search/listing.

POST http://localhost:8080/api/properties

Params Authorization Headers (9) Body Pre-request Script Tests Settings

none form-data x-www-form-urlencoded raw binary JSON

```
"ownerId": 8,
"title": "2BHK Apartment in Tenkasi",
"description": "Near bus stand, fully furnished",
"propertyType": "APARTMENT",
"address": "12, Lake View Road, Tenkasi",
"city": "Tenkasi",
"bedrooms": 2,
"bathrooms": 2,
"areaSqft": 1100,
"monthlyRent": 15000.00
```

Body Cookies Headers (11) Test Results Status: 201 Created

Pretty Raw Preview Visualize JSON

```
{
    "id": 8,
    "ownerId": 8,
    "ownerName": "Test Owner1",
    "title": "2BHK Apartment in Tenkasi",
    "description": "Near bus stand, fully furnished",
    "propertyType": "APARTMENT",
    "status": "AVAILABLE",
    "address": "12, Lake View Road, Tenkasi",
    "city": "Tenkasi",
    "bedrooms": 2,
```

GET http://localhost:8080/api/properties?city=Tenkasi&propertyType=APARTMENT&minRent=10000&maxRent=30000&page=0&size=5&sort=monthlyR

Params Authorization Headers (9) Body Pre-request Script Tests Settings

page	0
size	5
sort	monthlyRent,asc

Body Cookies Headers (11) Test Results Status: 200 OK Time: 1466 ms Size: 1.16 KB

Pretty Raw Preview Visualize JSON

```
    {
        "id": 1,
        "ownerId": 1,
        "ownerName": "Suresh Kumar",
        "title": "2BHK Apartment in Tenkasi",
        "description": "Near Courtallam, fully furnished",
        "propertyType": "APARTMENT",
        "status": "RENTED",
        "address": "12, Lake View Road, Tenkasi",
        "city": "Tenkasi",
        "bedrooms": 2,
        "bathrooms": 2,
        "areaSqft": 1100.0,
        "monthlyRent": 15000.00,
        "createdAt": "2026-10-07T11:21:15.966619",
        "updatedAt": "2026-10-08T11:41:43.718266"
    },
```

15. Tenant creation and tenant listing.

POST http://localhost:8080/api/tenants

Params Authorization Headers (8) Body Pre-request Script Tests Settings

none form-data x-www-form-urlencoded raw binary JSON

```
{
  "name": "Ajith",
  "phone": "8572596181",
  "email": "Ajith@gmail.com",
  "address": "ECR, CHENNAI",
  "occupation": "Racer"
}
```

GET http://localhost:8080/api/tenants

Params Authorization Headers (9) Body Pre-request Script Tests Settings

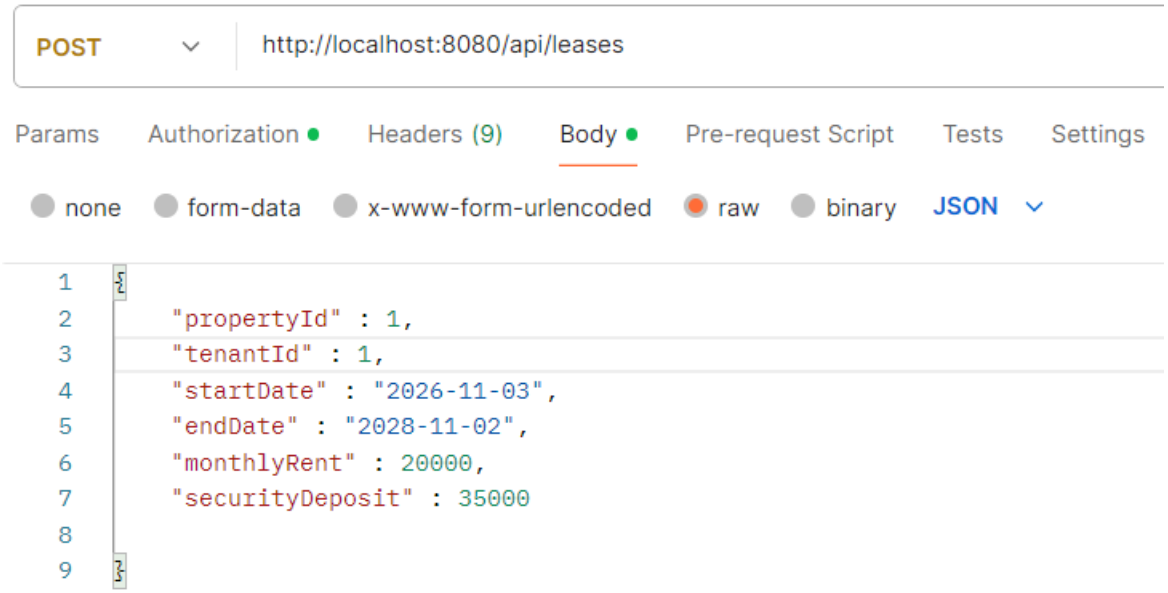
Type Bearer Token Token eyJhbGciOiJIUzM4NCJ9.eyJzdWIiOiJhZG1p...

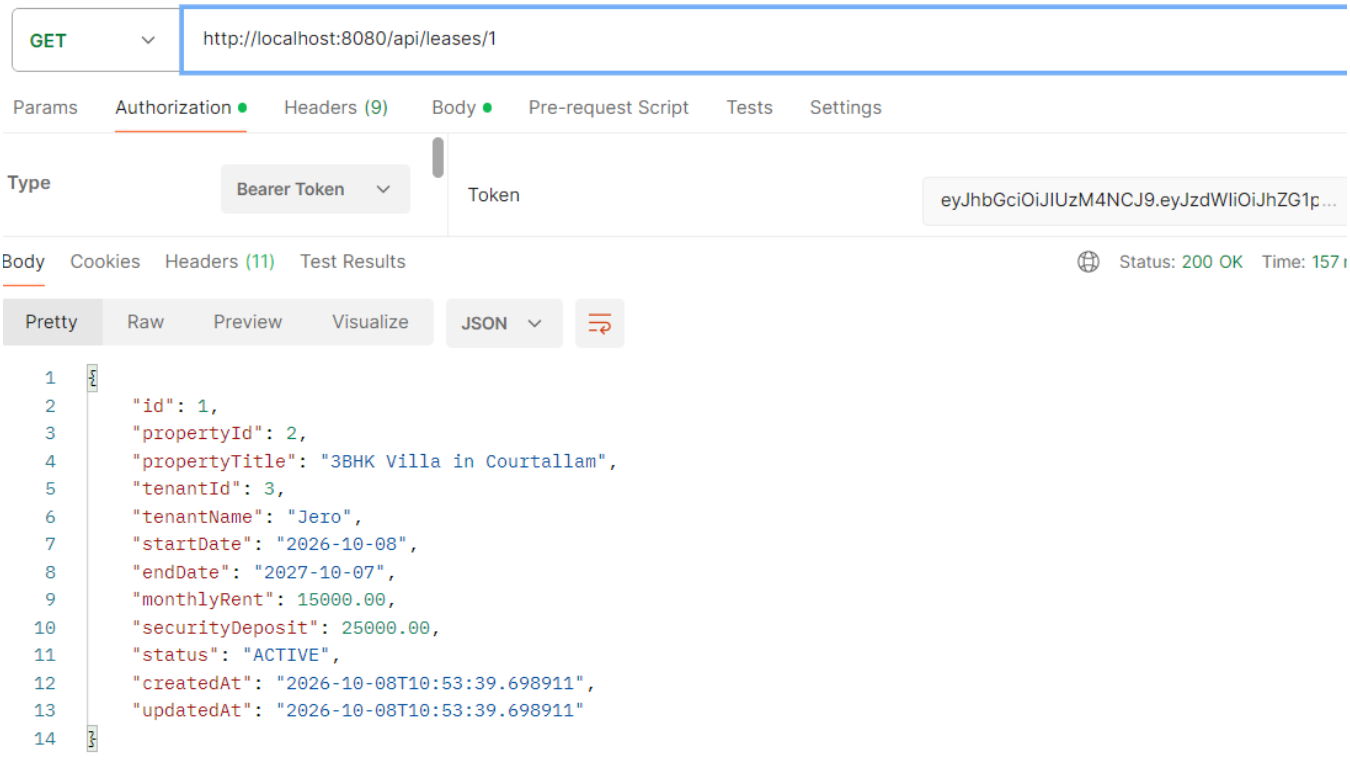
Body Cookies Headers (11) Test Results Status: 200 OK Time: 107 ms

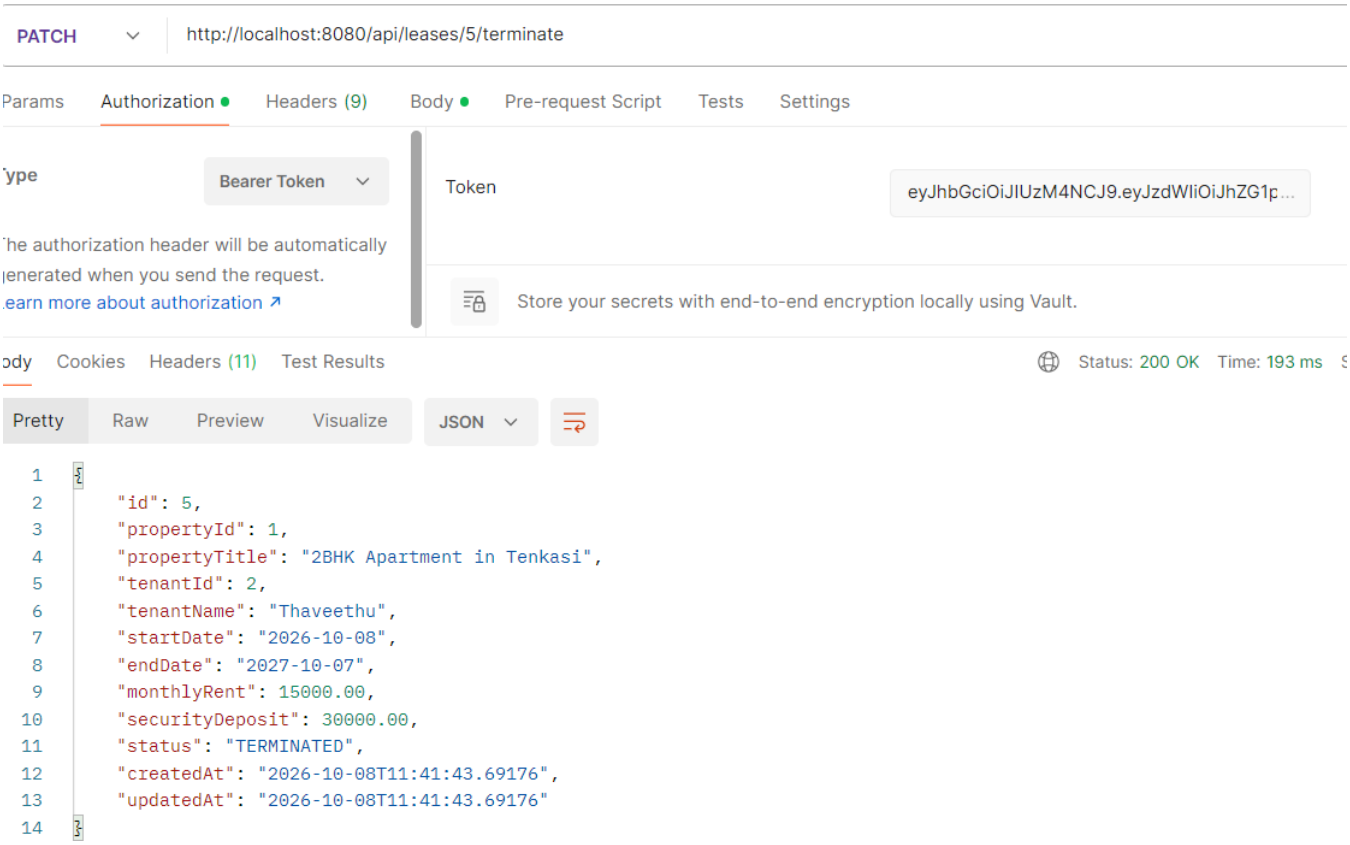
Pretty Raw Preview Visualize JSON


```
{
    "content": [
        {
            "id": 1,
            "name": "Ravi Kumar",
            "phone": "9876500001",
            "email": "ravi@gmail.com",
            "address": "T Nagar, Chennai",
            "occupation": "Software Engineer",
            "createdAt": "2026-10-08T09:58:16.521719",
            "updatedAt": "2026-10-08T09:58:16.521719"
        },
        {
            "id": 2,
            "name": "Thaveethu",
            "phone": "9457896181",
            "email": "Thaveethu@gmail.com",
            "address": "Puliyangudi, Tenkasi",
            "occupation": "Software Engineer",
            "createdAt": "2026-10-08T10:01:50.812689",
```

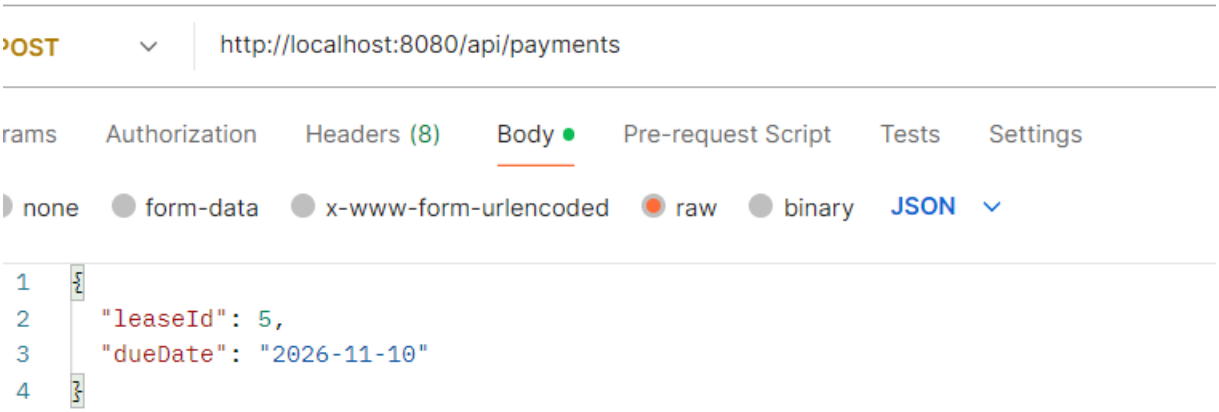
16. Lease creation, lease details, and lease termination.







17. Rent-payment creation, payment update, and summary.



PATCH http://localhost:8080/api/payments/1/pay

Params Authorization Headers (9) Body Pre-request Script Tests Settings

none form-data x-www-form-urlencoded raw binary JSON

```
{
  "paymentMethod": "CASH"
}
```

GET http://localhost:8080/api/payments/summary?leaseId=1

Params Authorization Headers (9) Body Pre-request Script Tests Settings

Query Params

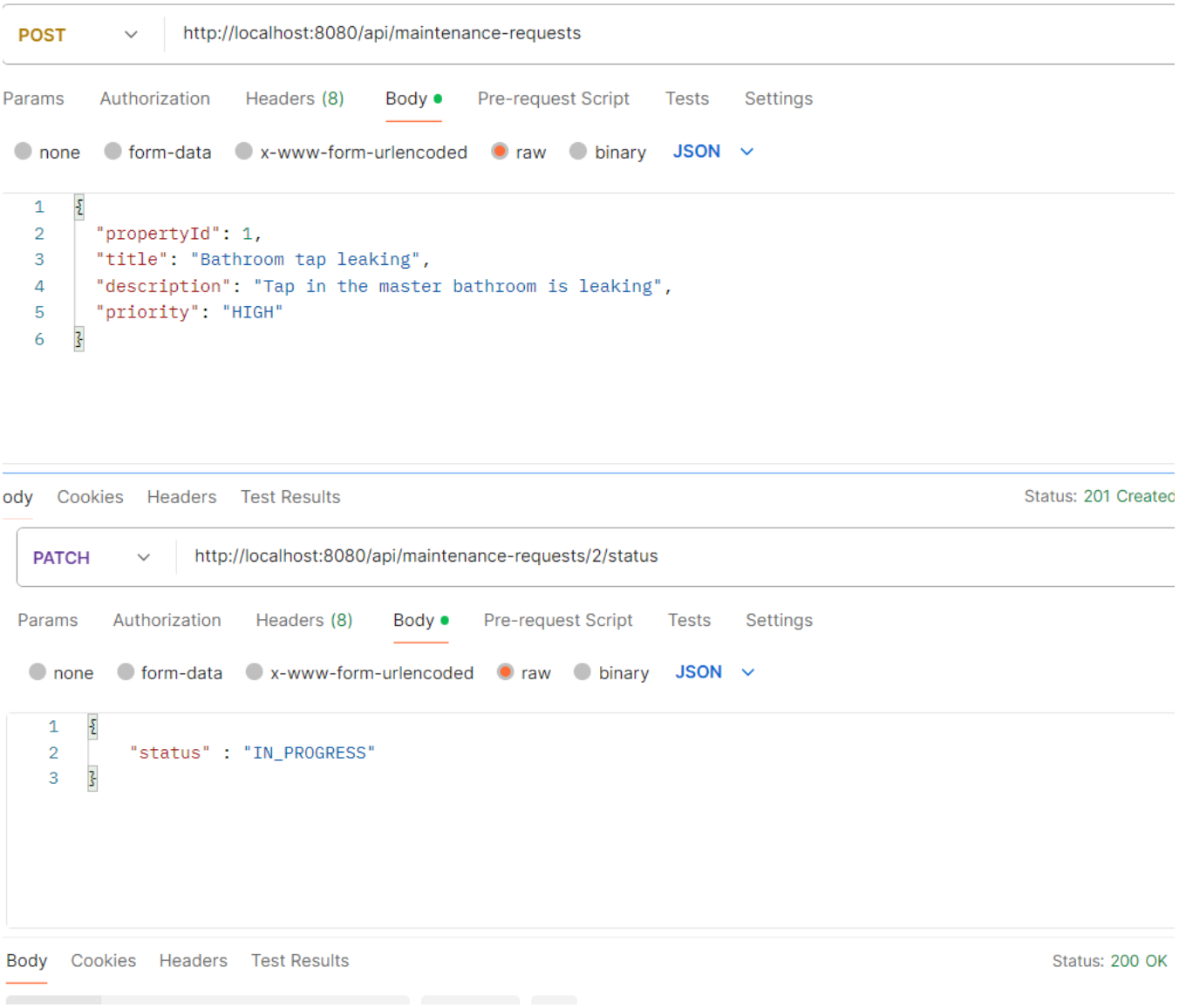
Key	Value
leaseId	1

Body Cookies Headers (11) Test Results Status: 200 OK

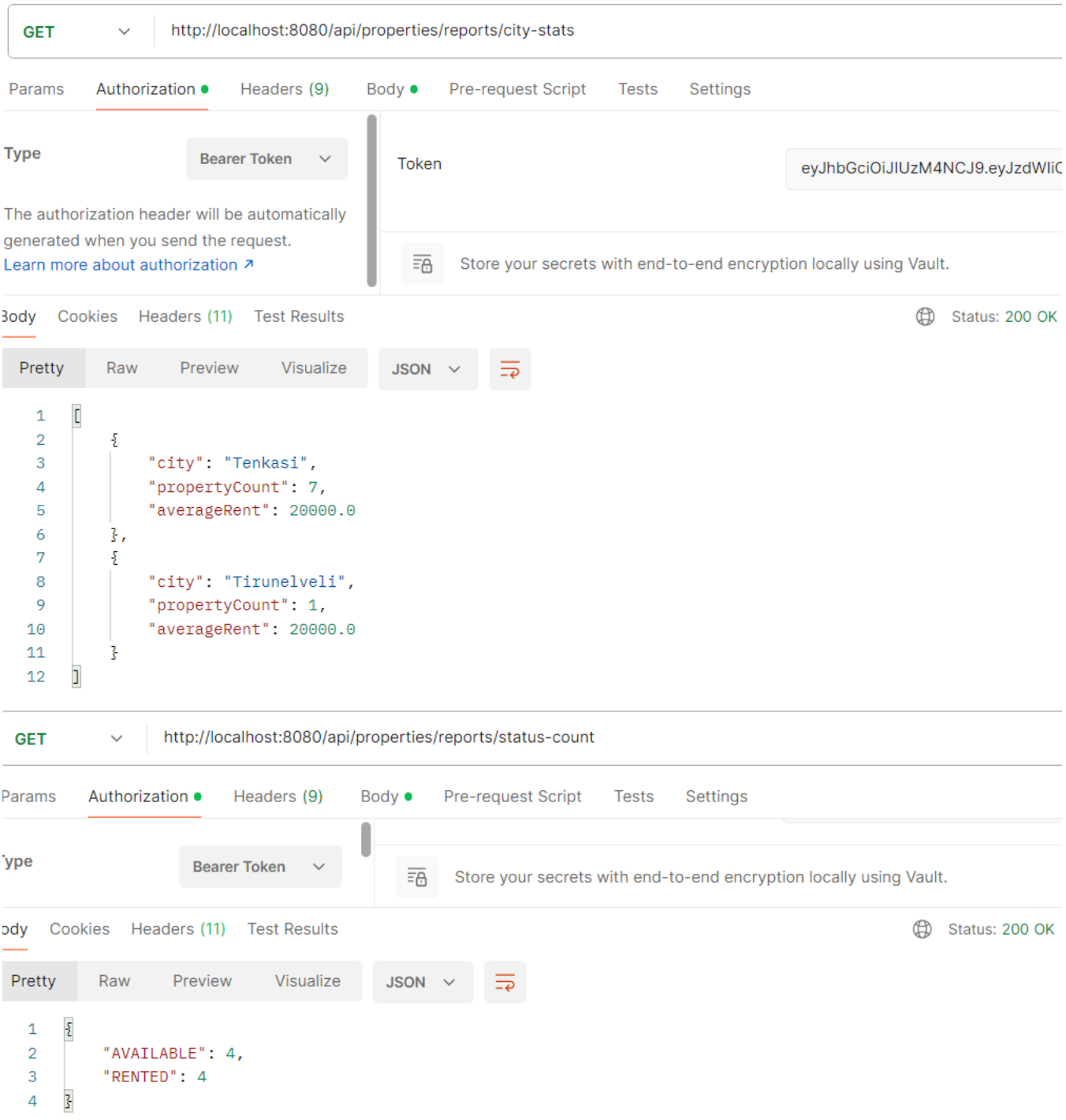
Pretty Raw Preview Visualize JSON

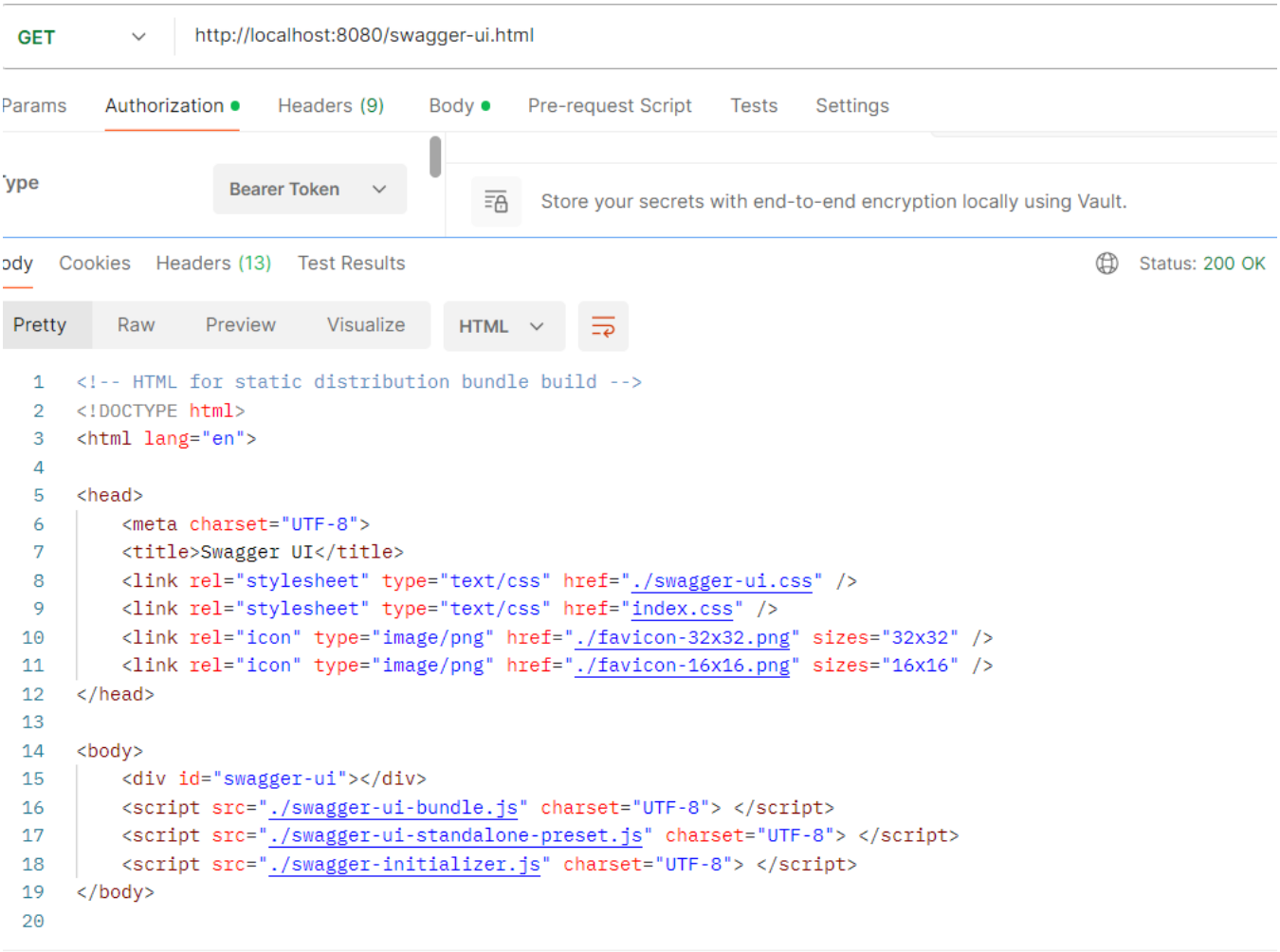
```
{
    "leaseId": 1,
    "totalPaid": 15000.00,
    "totalPending": 0,
    "totalOverdue": 0,
    "paidCount": 1,
    "pendingCount": 0,
    "overdueCount": 0
}
```

18. Maintenance request creation and status update.



19. City statistics/status-count report and Swagger UI.





18. Future Enhancements

- Develop a frontend dashboard for owners, tenants, and administrators.
- Add notification delivery for lease-expiry reminders and overdue rent.
- Add document upload and storage for lease agreements and receipts.
- Add automated integration tests for major workflows and authorization rules.
- Use environment-based configuration and secret management for production deployment.
- Add dashboards and exportable reports for rental income, occupancy, and maintenance costs.
- Deploy the application with monitoring, backups, and a documented release process.

19. Project Summary

The Real Estate Management System provides a REST-based backend for organizing property, owner, tenant, lease, rent-payment, and maintenance information. Built with Java and Spring Boot, it uses Spring Data JPA/Hibernate and MySQL for persistence, DTOs and mappers for data transfer, validation and centralized exception handling for request processing, and JWT-related security components for authentication. Reporting endpoints, pagination/filtering support, and scheduled lease/payment tasks extend the system beyond basic CRUD operations. The project can be further improved through a frontend, automated tests, secure deployment configuration, and additional reporting features.

End of Project Documentation